до 11 янв., 01:30

Открыть урок

Когда: 14.10.26 в 16:00

Урок 3. Условный оператор и не только — как заставить код принимать решения. Методический материал.

- Общая информация об уроке
- Структура урока
- Условный оператор if — else
- ✅ Задача «Котик голоден»
- Тип bool и операторы отношения
- ✅ Задача «Погода — наше всё»
- Вложенные условия и elif
- Логические операторы not, and, or
- ✅ Задача «Рабочий диапазон»
- ✅ Задача «На Марс»
- Длинные условия, pass, ...
- ✅ Задача «Грачи прилетели»
- Оператор in и спецсимволы
- ✅ Задача «Я не робот!»
- ✅ Задача «Допуск к глажению»
- Оператор match — case
- ✅ Задача «Длинная дорога»
- Завершение урока
- Таблица задач урока

Общая информация об уроке

Тип урока: Новая тема
Предварительные знания учеников: Переменные, типы данных, арифметические операции, преобразование типов, ввод и вывод данных, f-строки (уроки 1–2)

На этом уроке ученики научатся:

- Использовать оператор `if – else` для ветвления программы
- Понимать тип данных `bool` и операторы отношения (==, !=, <, >, <=, >=)
- Применять логические операторы `not`, `and`, `or`
- Строить вложенные условия и использовать `elif`
- Разбивать длинные условия на несколько строк и сохранять части условия в переменные
- Использовать оператор `pass` и `...` как заглушки
- Проверять наличие подстроки в строке с помощью `in` и `not in`
- Использовать спецсимволы (`\n`, `\t`, `\\`) и экранирование кавычек
- Применять оператор `match – case` для множественного выбора

Структура урока

№	Блок	Ориентировочное время	Задачи
1	Условный оператор `if – else`	8 – 10 мин	—
2	**Задача «Котик голоден»**	5 – 8 мин	Простое `if – else`, арифметика
3	Вложенные условия и `elif`	5 – 8 мин	—
4	Тип `bool` и операторы отношения	5 – 8 мин	—
5	**Задача «Погода — наше всё»**	5 – 8 мин	and/or, строки
6	Логические операторы `not`, `and`, `or`	4 – 6 мин	—
7	**Задача «Рабочий диапазон»**	5 – 8 мин	Двойные операторы отношения, and
8	**Задача «На Марс»**	8 – 10 мин	Комбинация `if – elif – else`, арифметика
9	Длинные условия, `pass`, `...`	4 – 6 мин	—

№	Блок	Ориентировочное время	Задачи
10	**Задача «Грачи прилетели»**	8 – 10 мин	Сложное составное условие
11	Оператор `in`, спецсимволы	5 – 8 мин	—
12	**Задача «Я не робот!»**	4 – 6 мин	`in`, логические операторы
13	**Задача «Допуск к глажению»**	5 – 8 мин	`in`, сравнение строк, `len()`
14	Оператор `match – case`	4 – 6 мин	—
15	**Задача «Длинная дорога»**	8 – 10 мин	`in`, `not in`, `len()`, комбинации
16	Подведение итогов	2–3 мин	—

⏱ **Темп урока:** Материала много. Задачи «Грачи прилетели», «Допуск к глажению» и «Длинная дорога» могут стать буфером для быстрых учеников или перейти в домашнее задание. Следите за темпом группы.

Условный оператор `if – else`

Время: 8 – 10 минут. Это центральная тема урока — не торопитесь здесь, остальные блоки опираются на этот фундамент.

Ключевые идеи для объяснения

- До этого момента программы были линейными — команды выполнялись строго по порядку. `if – else` — это первый инструмент, который позволяет программе «принимать решения».
- Метафора из учебника хорошо работает: `if – else` — это развилка в квесте. Можно использовать её как опорный образ на весь урок.
- **Ключевой синтаксис:**
  - После `if` и `else` **обязательно двоеточие**
  - Вложенный код **обязательно с отступом в 4 пробела**
  - `else`-часть необязательна; но если она есть, в ней нет смысла без `if`

```
weather = input()
if weather == "дождь":
```

```
    print("Бери зонт и резиновых уточек! 🐤")
else:
    print("Солнцезащитные очки — твой выбор!")
print("Хорошего дня!")
```

- Обратите особое внимание на **последнюю строку**: она выполняется в любом случае, потому что стоит без отступа. Это частый источник ошибок у учеников.

Рекомендация: Напишите пример живьём и намеренно допустите ошибку с отступом — перенесите `print("Хорошего дня!")` под `if` с отступом. Пусть ученики сами объяснят, что изменилось в поведении программы. Важно, чтобы они увидели: отступ — это не оформление, это логика.
`if` и `else` — блочные операторы. Правило одно для всех: строка оператора заканчивается двоеточием, вложенный код — с отступом 4 пробела. После блочного оператора должна быть хотя бы одна строка кода, но может быть и несколько, в том числе другие блочные операторы.

✅ Задача «Котик голоден»

Классная **Что проверяется:** простая конструкция `if – else`; арифметика — вычисление разницы двух чисел; преобразование ввода в `int`.

Как вести на уроке:

- Ключевой вопрос перед решением: «Как вычислить, сколько прошло времени?» — вычесть первое число из второго.
- Условие: если разница > 300 (5 минут = 300 секунд), то голоден.
- Напомните об `int(input())` — оба числа нужно приводить к целому.
- Типичная ошибка: перепутать порядок вычитания или использовать неправильный знак сравнения (>= вместо >). Проверьте граничный случай: ровно 300 секунд — это ещё «Выдохнули...».

Тип `bool` и операторы отношения

Время: 5 – 8 минут.

Ключевые идеи для объяснения

- Любой оператор отношения возвращает значение типа `bool`: `True` или `False`. Это можно показать прямо в `print`:

```
print(2 == 2)    # True
print(3 < 1)     # False
print("Привет" == 'привет')  # False — регистр важен!
```

- Шесть операторов отношения: ==, !=, <, >, <=, >=. **Особо** выделите == (сравнение) против = (присваивание) — это одна из самых распространённых ошибок.
- Python поддерживает **двойные операторы отношения** — редкая особенность языка:

```
if 0 <= x_coord < 100:
    print('В диапазоне')
```

- bool-значения можно задавать явно (`is_ready = True`) и использовать напрямую в `if`:

```
if is_cat_awake:
    print("😸 Мур-мур!")
```

- В Python почти всё приводится к `bool`: `False` — это 0 и пустая строка; `True` — всё остальное.

Рекомендация: Покажите таблицу всех шести операторов, дайте 2–3 минуты на то, чтобы ученики сами угадали результат нескольких выражений. Небольшая игра «Угадай вывод» хорошо запускает активное внимание.

✅ Задача «Погода — наше всё»

Классная **Что проверяется:** сравнение строк (==); логические операторы and, or; три ветки (`if – elif – else`); внимательность к регистру строк.

Как вести на уроке:

- Разберите условие вслух: нарисуйте таблицу сезон × осадки на доске, заполните «Нормально!» / «Глобальное изменение климата!» / «Не хватает данных.»
- Подсветите: строки вводятся строчными буквами (`лето`, `зима`, `дождь`, `снег`), регистр важен.
- Вариант реализации через `match – case` тоже допустим — его можно показать как бонус в конце урока.

Вложенные условия и `elif`

Время: 5 – 8 минут.

Ключевые идеи для объяснения

- Внутри `if`/`else` может находиться ещё один `if – else`. Каждый уровень вложенности добавляет 4 пробела к отступу.
- Проблема глубокой вложенности: код «убегает» вправо, читать невозможно. Решение — `elif`:

```
age = int(input())
if age >= 3000:
    print("О, великий древний!")
elif age >= 100:
    print("Добро пожаловать в музей COBOL.")
else:
    print("Юнец, иди учить PEP8!")
```

- После `if` может идти любое количество `elif`. Условия проверяются **сверху вниз**, и как только одно истинно — остальные не проверяются.

Рекомендация: Покажите один и тот же пример сначала через вложенные `if`, а потом через `elif` — наглядно демонстрирует, зачем нужен `elif`. Обратите внимание: порядок `elif` имеет значение. Если написать условие `>= 100` раньше `>= 3000`, вторая ветка никогда не выполнится.

Логические операторы `not`, `and`, `or`

Время: 4 – 6 минут.

Ключевые идеи для объяснения

Три оператора в порядке убывания приоритета:

- **not** — инвертирует: `True` → `False` и наоборот. Выполняется первым.
- **and** — оба должны быть истинны.
- **or** — достаточно хотя бы одного истинного.

Для повышения приоритета — **скобки**, как в математике.

Полезный приём — записывать части условия в переменные заранее. Это делает код читаемым:

```
is_roman = brother1 == 'Ромул' and brother2 == 'Рем'
is_scandinavian = brother1 == 'Тор' and brother2 == 'Локи'

if is_roman or is_scandinavian:
    print('✅ Знаток!')
```

Также покажите, что иногда **перефразирование условия** убирает вложенность и делает код проще: вместо «если всё хорошо → ура» можно написать «если что-то плохо → ошибка, иначе → ура».

Рекомендация: Обратите внимание на пример с записью результата сравнения в переменную:

```
is_rain = input() == 'да'
```

Поясните порядок выполнения: сначала `input()`, потом сравнение с `'да'`, потом присваивание. Это может выглядеть непривычно, но работает корректно и часто используется.

✅ Задача «Рабочий диапазон»

Классная **Что проверяется:** двойные операторы сравнения или and; логика «габариты коробки вписываются в окно»; вывод булева значения (`True`/`False`).

Как вести на уроке:

- Разберите условие: у окна две стороны (W, H), у коробки три стороны (a, b, c). Нужно понять, какие два измерения «торчат» из окна. Самый простой подход: взять максимальный и средний габарит коробки и проверить, вписываются ли они в окно (с учётом поворота).
- Более простой путь: перебрать, какие два из трёх измерений коробки могут быть «в плоскости» — например, отсортировать оба набора.
- Обратите внимание: вывод — `True` или `False` (с заглавной буквы!). Нельзя написать `print("True")` — это строка, а не булево значение. Нужно `print(True)` или условный вывод.
- Это хорошая задача для обсуждения: как ребята будут рассуждать о геометрии задачи? Попросите вслух проговорить алгоритм до написания кода.

✅ Задача «На Марс»

Классная **Что проверяется:** проверка на отрицательные числа; вычисление номера «стартового окна» через арифметику; конструкция `if – elif – else`; обработка граничных случаев.

Как вести на уроке:

- Сначала обработать спецслучай: если хотя бы одно из введённых чисел отрицательное — вывести `-1` и не делать ничего больше.
- Затем рассчитать: если дней до ближайшего окна больше или равно дням подготовки — `0`. Если нет — смотрим следующее окно. Формально: нужно найти минимальное `n`, такое что `days_to_window + 780 * n >= prep_days`, то есть `n = ceil((prep_days - days_to_window) / 780)`.
- Граничный случай: дней подготовки ровно столько, сколько до ближайшего окна — это `0`.

Длинные условия, `pass`, `...`

Время: 4 – 6 минут.

Ключевые идеи для объяснения

- Если условие не вмещается в строку — используем **круглые скобки** для переноса (PEP8 рекомендует выравнивать продолжение на 8 пробелов от начала `if`):

```
if (user_age > 12 and
        user_balance >= 1000 and
        not is_banned):
    print("Доступ разрешён!")
```

- Когда содержимое блока ещё не придумано или намеренно пусто — используется **pass**:

```
if user_status == "admin":
    pass  # TODO: добавить права
else:
    print("Доступ запрещён!")
```

- Альтернатива — `...` (`ellipsis`). Некоторые команды договариваются: `pass` = «так и задумано», `...` = «здесь будет код позже».

Рекомендация: `pass` лучше всего объяснять не абстрактно, а в контексте: «Представьте, что вы только набросали структуру программы и хотите запустить её, чтобы проверить одну ветку, а остальные заглушить». Покажите, что без `pass` Python упадёт с `IndentationError`.

✅ Задача «Грачи прилетели»

Классная / Домашняя **Что проверяется:** чтение пяти значений с отдельных строк; составное условие с `and`; проверка разниц между соседними днями (достаточно хотя бы одной пары с `or`); навык разбиения сложного условия на части.

Как вести на уроке:

- Разберите условие по частям вслух: (1) все пять температур > 10; (2) хотя бы одна пара соседних дней с разницей ≤ 2.
- Отдельно обсудите фразу «хотя бы между какими-нибудь соседними днями» — это `or` по всем четырём парам.
- Рекомендуйте сохранять части условия в булевы переменные (`all_above_10`, `close_enough`), а потом комбинировать:

```
all_above_10 = t1 > 10 and t2 > 10 and t3 > 10 and t4 > 10 and t5 > 1
close_enough = (abs(t1 - t2) <= 2 or abs(t2 - t3) <= 2 or
                abs(t3 - t4) <= 2 or abs(t4 - t5) <= 2)
if all_above_10 and close_enough:
    print("Пора на север!")
else:
    print("Ещё не пора.")
```

- Задача может перейти в домашнее, если времени не хватает.

Оператор `in` и спецсимволы

Время: 5 – 8 минут.

Ключевые идеи для объяснения

Оператор `in` проверяет, является ли строка подстрокой другой:

```
text = input()
if "кот-учёный" in text:
    print("Найден кот-ботаник!")
```

- Для проверки отсутствия — `not in` (не `not (... in ...)`).
- `in` будет встречаться снова, когда перейдёте к спискам и другим коллекциям — это стоит упомянуть.

Спецсимволы:

- `\n` — перенос строки
- `\t` — табуляция
- `\\` — обратный слеш
- `\"` или `\'` — кавычка внутри той же кавычки

```
print("Первая строка\nВторая строка\tТабуляция")
print("Хочу пиццу \"Маргариту\"")
print("Путь: C:\\Users\\Кот")
```

Рекомендация: Покажите `\n` в `print` и сравните с двумя отдельными `print` — результат одинаковый, но это разные способы. Для спецсимволов достаточно показать 2–3 примера, акцент — на `\n` и экранировании кавычек, они встречаются чаще всего.

✅ Задача «Я не робот!»

Классная **Что проверяется:** оператор `in` для трёх подстрок; логическое `or`.

Как вести на уроке:

- Задача простая и короткая — хорошо подходит для закрепления `in` сразу после объяснения.
- Напомните: `in` чувствителен к регистру. «Пере» и «пере» — разные строки.
- Решение в одну строку условия: `if "пере" in text or "ход" in text or "зебра" in text`.

✅ Задача «Допуск к глажению»

Классная / Домашняя **Что проверяется:** `in` для трёх подстрок; сравнение строк (`>=`); `len()` для строк; комбинация `and`/`or`; понимание лексикографического порядка.

Как вести на уроке:

- Объясните **лексикографическое сравнение**: строки сравниваются посимвольно, как в словаре. `'б' > 'а'`, `'кот' > 'коза'`.
- Разберите условие по частям: (1) длина проверяемой строки ≥ длины образца (`len(s2) >= len(s1)`); (2) проверяемая строка лексикографически ≥ образца (`s2 >= s1`); (3) в строке есть `'Мур'` или `'мур'` или `'хрр'`.
- Все три части объединяются через `and`, третья часть внутри — через `or`.
- Вывод — `True`/`False` через `print`, как и в «Рабочем диапазоне».
- Задача может перейти в домашнее задание.

Оператор `match – case`

Время: 4 – 6 минут. Блок скорее обзорный — применение `match – case` не является обязательным на этом уроке.

Ключевые идеи для объяснения

`match – case` — аналог `switch` из других языков. Удобен, когда одна переменная проверяется на много значений:

```
day = input()
match day:
    case "понедельник":
        print("Начало недели...")
    case "суббота" | "воскресенье":
        print("Выходные! 🎉")
    case _:
        print("Любой другой день.")
```

- `|` внутри `case` — это «или».
- `_` — дефолтный кейс (как `else`).
- Каждый блок `case` может содержать любое количество строк кода.

Рекомендация: Подчеркните, что `match – case` — это не просто синтаксический сахар: в курсе к нему ещё вернутся, и его возможности значительно шире простого перебора значений. Сейчас задача — познакомиться с синтаксисом.

✅ Задача «Длинная дорога»

Домашняя / Дополнительная **Что проверяется:** `in` и `not in` для проверки вхождения символа в строку; `len()` для нахождения длины; комбинирование условий; нахождение максимума среди отфильтрованных значений.

Как вести на уроке:

- Разберите алгоритм: для каждой из трёх дорог проверить, что обязательный город (`must`) есть в строке, а запрещённый (`must_not`) — нет. Из подходящих дорог выбрать самую длинную.
- Хороший момент для обсуждения: как найти максимум среди условно подходящих? Можно собирать кандидатов в переменные и сравнивать. Или начать с `result = 0` и обновлять.
- Напомните: `in` проверяет вхождение **подстроки** в строку, то есть `'D' in 'ABCDFENHG'` — это `True`.
- Если ни одна дорога не подошла — вывести `0`.

- Задача хорошо подходит в качестве домашней или дополнительной для сильных учеников.

Завершение урока

Время: 2–3 минуты.

Что обязательно сказать

- Сегодня программы стали «умнее» — они умеют принимать решения. Это основа почти любой реальной программы.
- Напомнить про **классные и домашние задачи**: если что-то не успели — продолжить дома.
- **Домашнее задание:** дочитать урок в учебнике, выполнить домашние задачи в ЛМС.
- Напомнить, где задавать вопросы: чат задачи в ЛМС и групповой чат.

Таблица задач урока

Классная работа

Задача	Баллы	Что проверяется	На что обратить особое внимание
Котик голоден	10	Простой `if – else`; вычисление разницы двух чисел; `int(input())`	Граничный случай: ровно 300 секунд — это «Выдохнули...» (условие строгое >). Не перепутать порядок вычитания
Погода — наше всё	10	Сравнение строк; and/or; три ветки `if – elif – else`	Регистр строк важен (`лето`, а не `Лето`). Разобрать таблицу сезон × осадки перед кодированием
Рабочий диапазон	10	Двойные операторы сравнения или and; геометрическая логика	Вывод — `True`/`False` без кавычек. Нужно правильно выбрать, какие два из трёх измерений сопоставлять с окном
На Марс	15	`if – elif – else`; проверка отрицательных чисел;	Спецслучай с отрицательными числами — первым `if`. Граничный случай: дней

Задача	Баллы	Что проверяется	На что обратить особое внимание
		арифметика с остатком/делением	подготовки = дням до окна → ответ `0`
Грачи прилетели	15	Пять `int(input())`; составное `and` по всем температурам; `or` по парам соседних дней; `abs()`	«Хотя бы одна пара» — это `or`, а не `and`. Рекомендовать разбить на две булевы переменные
Я не робот!	10	`in` для трёх подстрок; `or`	Регистр чувствителен: «Пере» ≠ «пере». Решение в одно условие
Допуск к глажению	15	`in` для трёх подстрок; `len()`; лексикографическое сравнение `>=`; `and`/`or`	Объяснить лексикографический порядок. Вывод `True`/`False` без кавычек
Длинная дорога	15	`in` / `not in` для символов в строке; `len()`; нахождение максимума среди отфильтрованных	`in` ищет вхождение, `len()` считает длину. Если ни одна дорога не подошла — вывести `0`

Домашние задачи

Задача	Баллы	Что проверяется	На что обратить особое внимание
Влажность	15	Четыре диапазона числовых значений; `if – elif – else`; двойные операторы сравнения	Границы диапазонов включительно (`<=`). Порядок `elif` важен: проверять критический диапазон (`> 75`) до просто «влажного» (`> 60`), иначе ветка с «Критически влажно!» никогда не выполнится
Двенадцать месяцев	15	`match – case` с `|`; 12 вариантов ввода + дефолтный `case _`; вывод на английском	Хороший повод применить `match – case` по назначению. Напомнить: `case "December" | "January" | "February"` — несколько значений в одном `case`. Вариант «Not defined.» — с точкой в конце, точно по условию
Шелест дождя	20	`in` с подстроками в разных регистрах; три ветки: нет ш/ст, есть ш/ст без р, есть и то и другое	Сложное условие из-за поиска подстрок в разных регистрах. Логика трёх веток — проверить наличие «шума» и наличие «р» независимо, затем скомбинировать. Для удобства

Задача	Баллы	Что проверяется	На что обратить особое внимание
			части условия записать в отдельные переменные
Шалость удалась	20	`min()` и `max()` для символов строки; сравнение символов; логика «крест-накрест»	Функции `min(s)` и `max(s)` работают со строками, возвращая минимальный/максимальный символ. Два сценария совпадения: `min(s1) == max(s2)` **или** `max(s1) == min(s2)`
Второй фактор	30	Вложенные условия; счётчик попыток; логика многоступенчатой аутентификации; `if – elif – else`	Задача проверяется преподавателем вручную. Обратить внимание на корректный счётчик попыток и правильные сообщения об оставшихся попытках. Пароль и код подтверждения — произвольные, но должны быть захардкожены в коде